

冲刺 NOI2026 省选联合模拟试题

BSZX

时间：2025 年 12 月 20 日 08:00 ~ 13:00

题目名称	加油站	美感数组	音乐游戏
题目类型	传统型	传统型	传统型
目录	stations	array	game
可执行文件名	stations	array	game
输入文件名	stations.in	array.in	game.in
输出文件名	stations.out	array.out	game.out
每个测试点时限	2.0 秒	2.0 秒	2.0 秒
内存限制	1024 MiB	1024 MiB	512 MiB
测试点数目	6	6	4
测试点是否等分	否	否	否

提交源程序文件名

对于 C++ 语言	stations.cpp	array.cpp	game.cpp
-----------	--------------	-----------	----------

编译选项

对于 C++ 语言	-O2 -std=c++14 -static
-----------	------------------------

注意事项（请仔细阅读）

- 文件名（程序名和输入输出文件名）必须使用英文小写，不需要建立子文件夹。
- C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 `0`。
- 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
- 选手提交的程序源文件必须不大于 100KB。
- 程序可使用的栈空间内存限制与题目的内存限制一致。
- 若无特殊说明，输入文件与输出文件中同一行的相邻整数均使用一个空格分隔。
- 直接复制 PDF 题面中的多行样例，数据将带有行号，并且某些字符可能无法正常显示，建议选手直接使用对应目录下的样例文件进行测试。
- 评测时采用的机器配置为：Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz，内存 32GB。上述时限以此配置为准。

加油站 (stations)

【题目描述】

巴蜀国的高速公路网络由 N 个城市和 $N - 1$ 条路组成，每条路的长度（公里）已知。我们知道任意两个城市之间只有一条路径。此外，每个城市恰好有一个加油站，其他地方没有。

有一天，许多人决定开车旅行。总共有 N^2 辆车在路上。奇怪的是，对于每个有序城市对 (a, b) ，恰好有一辆车从城市 a 开往城市 b ，沿着这两个城市之间唯一的路径行驶。由于巴蜀人全都开保时捷汽车，每辆车的油箱容量都是 K 升，并且每公里消耗一升汽油。出发前，每辆车的油箱都是满的。此外，巴蜀人非常有规律。由于懒惰，他们只在汽油不足以到达下一个城市时才加油（油箱为空时可以进入城市）。一旦他们被迫停在加油站，他们总是把油箱加满。

巴蜀国税务部门想知道当天每个加油站停了多少辆车。鉴于这种可预测的行为，你应该能够轻松计算出来。

【输入格式】

从文件 `stations.in` 中读入数据。

输入的第一行包含两个空格分隔的整数 N 和 K ，分别表示城市数量和每辆车的油箱容量。

接下来的 $N - 1$ 行描述道路。每行包含三个空格分隔的整数 u_i, v_i, l_i ，其中 u_i 和 v_i 是第 i 条路连接的城市编号， l_i 是这条路的长度（公里）。城市从 0 到 $N - 1$ 编号。保证任意两个城市之间只有一条路径。

【输出格式】

输出到文件 `stations.out` 中。

输出 N 行，表示从城市 0 到城市 $N - 1$ 每个城市加油站停靠的汽车数。

【样例 1 输入】

```
1 3 1
2 0 1 1
3 1 2 1
```

【样例 1 输出】

```
1 0
2 2
3 0
```

【样例 1 解释】

有 3 座城市呈直线排列，连接它们的道路长度为 1，油箱容量为 1 升。只有在两个外围城市之间行驶的汽车会在中间城市加油。

【样例 2 输入】

```
1 6 2
2 0 1 1
3 1 2 1
4 2 3 1
5 3 4 2
6 4 5 1
```

【样例 2 输出】

```
1 0
2 3
3 3
4 12
5 8
6 0
```

【样例 2 解释】

有 6 个城市呈直线排列，油箱容量为 2 升。许多汽车需要在城市 3 和城市 4 加油。这是有道理的，因为城市 3 和城市 4 之间的道路长度为 2 公里。

【样例 3】

见选手目录下的 *stations/stations3.in* 与 *stations/stations3.ans*。
该样例满足子任务 1 的限制条件。

【样例 4】

见选手目录下的 *stations/stations4.in* 与 *stations/stations4.ans*。
该样例满足子任务 4 的限制条件。

【样例 5】

见选手目录下的 *stations/stations5.in* 与 *stations/stations5.ans*。
该样例满足子任务 5 的限制条件。

【样例 6】

见选手目录下的 *stations/stations6.in* 与 *stations/stations6.ans*。
该样例满足子任务 6 的限制条件。

【数据范围】

对于所有输入数据, 满足: $2 \leq N \leq 7 \times 10^4, 1 \leq K \leq 10^9, 0 \leq l_i \leq K (0 \leq i \leq N - 2)$
令 D 表示连接到单个城市的道路的最大数量。详细子任务附加限制及分值如下表所示。

子任务编号	$N \leq$	$K \leq$	特殊性质	分值
1	1000	1000	无	20
2	7×10^4	10^9	A,C	10
3			A	15
4		10	B	15
5			无	20
6		10^9		20

- 特殊性质 A: 满足 $D \leq 2$;
特殊性质 B: 满足 $D \leq 10$;
特殊性质 C: 满足 $l_i = 1 (0 \leq i \leq N - 2)$ 。

美感数组 (array)

【题目描述】

巴蜀国的人们普遍认为，一个整数数组 $a[0], a[1], a[2], \dots, a[m - 1]$ 的**美丽值**定义为：满足以下条件的正整数 k 的个数——你可以将该数组划分为 k 个互不重叠的子数组（即连续元素的序列），使得：

- 1. 每个元素恰好属于一个子数组；
- 2. 所有子数组具有相同的**最小缺失元素**。

这里，一个整数数组的**最小缺失元素**是指数组中没有出现的、严格大于 0 的最小正整数。

给定一个整数数组 $v[0], v[1], \dots, v[n - 1]$ ，以及 q 个询问，每个询问的形式为 (l_i, r_i) ，其中对所有 $0 \leq i < q$ ，均有 $1 \leq l_i \leq r_i \leq n$ 。

对于每个询问，你要求出数组 $v[l_i - 1], v[l_i], \dots, v[r_i - 1]$ 的美丽值。

【输入格式】

从文件 `array.in` 中读入数据。
第一行两个整数 n 和 q ，表示整数数组的大小和询问的数量。
第二行 n 个正整数，表示 $v[0], v[1], \dots, v[n - 1]$ 。
接下来 q 行，每行两个正整数 l_i, r_i ，表示一个询问区间。

【输出格式】

输出到文件 `array.out` 中。
输出共 q 行，每行一个整数，依次表示每个询问的答案。

【样例 1 输入】

```
1 10 2
2 1 1 2 2 3 3 1 2 3 4
3 1 6
4 1 9
```

【样例 1 输出】

```
1 1
2 2
```

【样例 1 解释】

对于第一个查询，我们只能将区间分成一个子数组，从位置 1 到位置 6。

对于第二个查询， k 可以是 1 或 2。

将区间分成 1 个子数组的一种可能是选择从位置 1 到位置 9 的子数组。将区间分成 2 个子数组的一种可能是选择从位置 1 到位置 6 和从位置 7 到位置 9 的子数组。

第一个查询的答案是 1，第二个查询的答案是 2。

【样例 2】

见选手目录下的 *array/array2.in* 与 *array/array2.ans*
该组样例满足子任务 2 的条件。

【样例 3】

见选手目录下的 *array/array3.in* 与 *array/array3.ans*
该组样例满足子任务 3 的条件。

【样例 4】

见选手目录下的 *array/array4.in* 与 *array/array4.ans*
该组样例满足子任务 4 的条件。

【样例 5】

见选手目录下的 *array/array5.in* 与 *array/array5.ans*
该组样例满足子任务 5 的条件。

【样例 6】

见选手目录下的 *array/array6.in* 与 *array/array6.ans*
该组样例满足子任务 6 的条件。

【数据范围】

对于所有输入数据，满足：

- $1 \leq n \leq 6 \times 10^5$;
- $1 \leq q \leq 6 \times 10^5$;
- 对于所有 $0 \leq i < n, 1 \leq v[i] \leq 4 \times 10^5$;
- 对于所有 $0 \leq i < q, 1 \leq l_i \leq r_i \leq n$;

详细子任务附加限制及分值如下表所示。

子任务编号	$n \leq$	$q \leq$	特殊限制	分值
1	10	100	无	10
2	100			15
3	1000			20
4	10^5	10^5	A	15
5	7.5×10^4	7.5×10^4	无	20
6	6×10^5	6×10^5		20

特殊性质 A：满足对于所有 $0 \leq i < n, 1 \leq v[i] \leq 2$ 。

音乐游戏 (game)

【题目描述】

你在打音游，这一关有 n 个音符，会从屏幕的最上方落下，你需要在屏幕最下方的线上打到这些音符。

判定线可以看作数轴的正半轴， n 个音符依次落下，第 i 个音符会落到 p_i 的位置。你的两只手可以分别覆盖两个长度为 L 的区间，准确来说，一只手可以覆盖 $[x, x + L]$ ，使得当音符落在这个区间（包括端点）的时候可以准确打到。

游戏开始时你的两只手覆盖了 $[0, L]$ ，在接下来的 n 个音符落下的间隙中，你可以将你的手任意移动，移动的时间忽略不计，从 $[x, x + L]$ 移动到 $[y, y + L]$ 需要耗费 $|x - y|$ 的体力。

游戏开始前，你想要知道最少耗费多少体力可以依次打到所有的音符。

【输入格式】

从文件 `game.in` 中读入数据。

第一行两个整数 n, L 。

第二行 n 个整数，第 i 个整数 p_i 表示第 i 个音符的位置。

【输出格式】

输出到文件 `game.out` 中。

输出一行一个整数表示最少耗费的体力。

【样例 1 输入】

```
1 4 1
2 6 3 7 1
```

【样例 1 输出】

```
1 9
```

【样例 1 解释】

两只手的移动分别是： $[0, 1] \rightarrow [5, 6] \rightarrow [6, 7]$ 和 $[0, 1] \rightarrow [2, 3] \rightarrow [1, 2]$ ，花费体力 $6 + 3 = 9$ 。

【样例 2】

见选手目录下的 *game/game2.in* 与 *game/game2.ans*。
该样例满足子任务 1 的限制。

【样例 3】

见选手目录下的 *game/game3.in* 与 *game/game3.ans*。
该样例满足子任务 2 的限制。

【样例 4】

见选手目录下的 *game/game4.in* 与 *game/game4.ans*。
该样例满足子任务 3 的限制。

【样例 5】

见选手目录下的 *game/game5.in* 与 *game/game5.ans*。
该样例满足子任务 4 的限制。

【数据范围】

对于 100% 满足 $1 \leq n \leq 50000, 0 \leq L \leq 50, 0 \leq p_i \leq 10^9$ 。

子任务编号	特殊限制	分值
1	$n \leq 200, p_i \leq 200$	15
2	$L = 0$	15
3	$L \leq 5$	30
4	无特殊限制	40